Birzeit University - Faculty of Engineering and Technology
Electrical & Computer Engineering Department - ENCS4330
Real-Time Applications & Embedded Systems - 2^{nd} semester - 2025/2026

Project #3
Sockets & POSIX threads under Linux
Due: June 10, 2026

Instructors: Dr. Ahmad Afaneh, Dr. Hanna Bullata

Distributed Software Update Framework using Sockets and Multi-threading

Modern software systems often require automaticc update mechanisms that allow client applications to communicate with remote servers, detect new so ware versions, and download updates without requiring manual intervention.

In this project, you will design and implement a distributed client/server software update framework using sockets and multithreading. The project focuses on building a reliable networked application capable of handling multiple concurrent clients while supporting automatic software update functionality.

The system should simulate a real-world update service where client applications connect to a centralized update server to verify whether newer software versions are available.

The implementation should demonstrate proper communication between distributed components while ensuring scalability and responsiveness.

The project can be explained as follows:

1. **Main System Behavior**: The system consists of two major components:
   - Update Server.
   - Client Application.

   The client application should connect to the server, send its currently installed software version, and determine whether a software update is required.

   The update server should maintain information about the latest available software version and respond to client requests appropriately.

   If the client software is outdated, the server should transmit the update package to the client. Otherwise, the client should be informed that it is already up to date.

   The system should support multiple simultaneous client connections without causing clients to block or wait for one another.

2. **Client Application Requirements**: The client application represents installed software running on a user machine. The client main function should invoke a function similar to:

   ```
   CheckForUpdate()
   ```

   The client should perform the following operations:
   - Establish a connection with the update server.
   - Send the current software version number to the server.
   - Wait for the server response.
   - Determine whether an update is available.
   - If an update exists:
     * Download the update file.

    - Store the received file locally.
    - Optionally execute or simulate execution of the update.
  - If no update is required:
    - Display an appropriate message.
    - Close the connection gracefully

3. **Server Application Requirements**: The server application should continuously listen for incoming client connections. The Server should:
   - Start listening on a predefined port.
   - Accept incoming client connections.
   - Create a separate thread for each connected client.
   - Handle all client communication independently.
   - Receive the client software version.
   - Compare the received version against the latest available version.
   - Notify the client whether an update is required.
   - Send the update file if needed.
   - Close the connection safely after completion.

To how much clients ?

   The server must be capable of serving multiple clients simultaneously. No client should wait for another client to finish its update operation.

4. **Communication Requirements**: Messages exchanged between the client and server should include enough information for proper interpretation.
   p1mm]If an update is required, the server should transfer the update package to the client. The client should save the received file locally.

5. **Version Management**: The server should maintain the latest software version information. You may store this information using:
   - Configuration files
   - Text files
   - Databases

   The client should use a function similar to:

   ```
   int getCurrentVersion()
   ```

   You do not need to implement actual software installation behavior. A simulation is sufficient.

6. **Monitoring and Logging**: The system should include monitoring and logging functionality. The log should record important events such as:
   - Client connection attempts
   - Successful connections
   - Disconnections
   - Version requests
   - Update decisions
   - Transfer completion
   - Failed downloads
   - Errors
   - Server startup and shutdown

   Each log entry should include:

- Timestamp
- Client information
- Event description
- Thread identifier if applicable

Logs may be:

- Written to files
- Displayed using a simple GUI

7. **Reliability and Error Handling**: The system should handle unexpected situations gracefully. The application should avoid crashing whenever possible.

8. **Testing Requirements**: You should test the system under different scenarios, including:

   - Single client connection
   - Multiple simultaneous clients
   - Clients with outdated versions
   - Clients already up to date
   - Interrupted connections
   - Large file transfers
   - Invalid client requests
   - Concurrent downloads

   You should verify that:

   - The server remains responsive
   - Multiple clients work correctly
   - File transfers complete successfully
   - No race conditions occur
   - Threads terminate properly

9. **Expected Deliverables**: You should submit Source code for the complete client/server system.

10. **Optional Advanced Features**: You may implement additional advanced features such as:

    - Secure encrypted communication
    - Authentication
    - GUI interface
    - Download resume support
    - Automatic retries
    - Distributed update mirrors
    - Load balancing
    - Checksum validation
    - Performance statistics

What you should do

- In order to implement the above-described application, you need to use the socket programming and POSIX threads techniques we've seen in class. Be wise in the choices you make and be ready to convince us that you made the best choices :-)
- Write the code for the above-described application using a multi-threading approach.
- In order to avoid hard-coding values in the code, think of creating a text file that contains user-defined values or ranges and pass the file name as an argument to the application so that new values can be loaded with each run. That will spare you from having to change your code permanently and re-compile.
- Use graphics elements from opengl library in order to best illustrate the application. Nothing fancy, just simple and elegant elements are enough.
- Test your program.
- Check that your program is bug-free. Use the `gdb` debugger in case you are having problems during writing the code (and most probably you will :-). In such a case, compile your code using the `-g` option of the `gcc`.
- The project lead is responsible to send the zipped folder that contains your source code and your executable before the deadline on behalf of the team. If the deadline is reached and you are still having problems with your code, just send it as is!